

Detailed Study of CROC SOC

Introduction

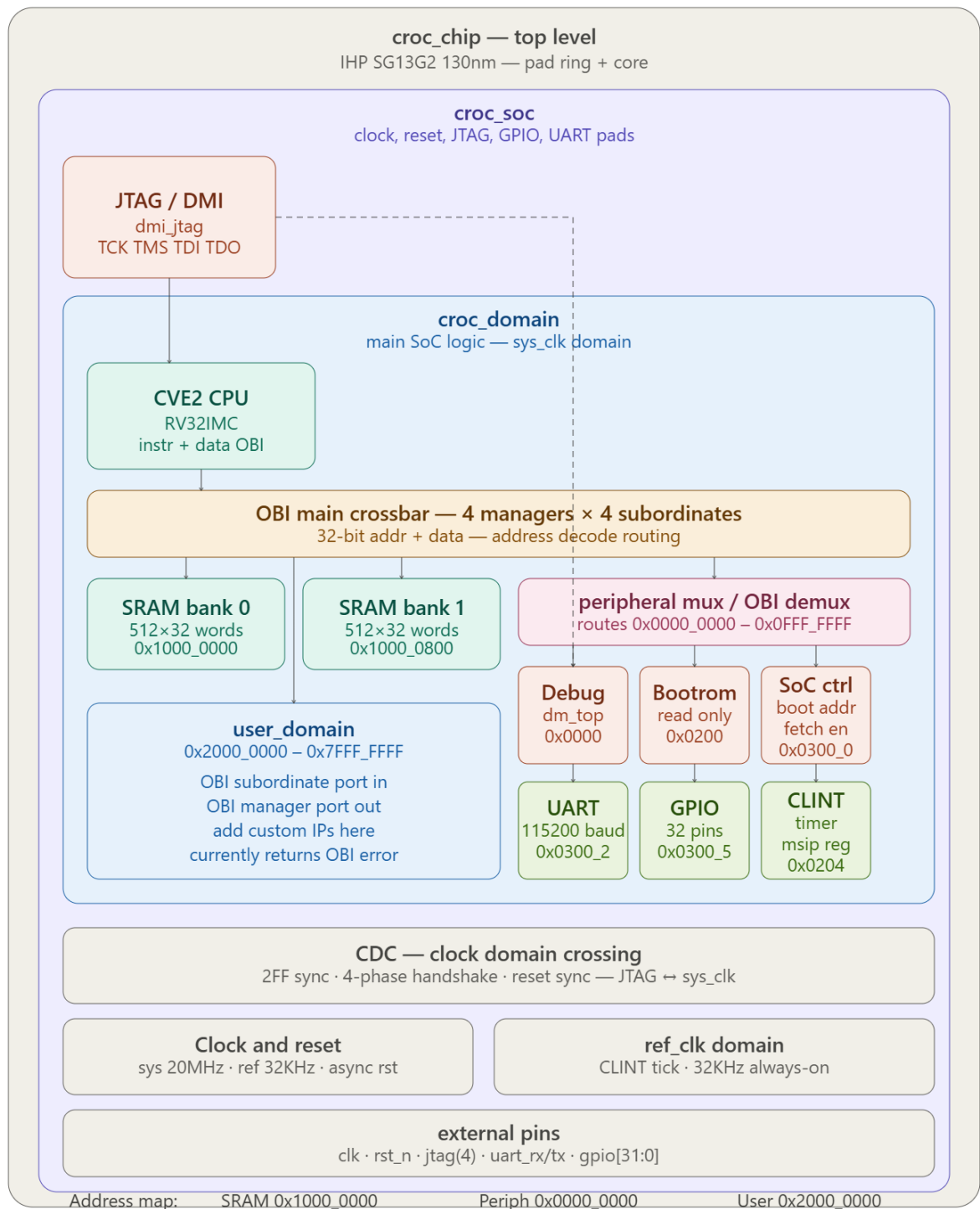
Croc is an open-source educational SoC developed by ETH Zurich and University of Bologna as part of the PULP platform. It implements a RISC-V RV32IMC CPU (CVE2/Ibex), OBI crossbar, SRAM, UART, GPIO, Timer and JTAG debug interface, targeting IHP's open-source 130nm technology.

Git and environment setup

- Opened GitHub Codespaces on the Croc repository — providing a free Linux-based cloud development environment
- Initialized Git submodules to fetch all IP dependencies including CVE2 CPU, OBI crossbar, iDMA and IHP PDK
- Learned key Git commands: clone, submodule update, status, log

SoC Architecture understanding

- CVE2 (Ibex) RISC-V CPU — 2-stage pipeline, RV32IMC
- OBI Crossbar — central bus routing all transactions by address
- Memory map — Bootrom at 0x0000_0000, SRAM at 0x1000_0000, peripherals at 0x2000_xxxx
- JTAG debug interface — TCK, TMS, TDI, TDO
- User domain — extension point for custom hardware at 0x5000_0000
- Boot sequence — CPU starts at Bootrom, copies program to SRAM, jumps and execute



Tools used

Tool	Version	Purpose
Git	2.x	Version control
Verilator	5.020	SystemVerilog simulation
Rust/Cargo	1.94.1	Required for bender
bender	0.31.0	Hardware IP dependency manager
IHP sg13g2 PDK	—	130nm process design kit

Compilation

The Croc SoC was compiled using Verilator 5.020, an open-source SystemVerilog to C++ simulator. The following steps were performed:

Dependency resolution: bender 0.31.0 was used to resolve 9 IP dependencies including the CVE2 RISC-V CPU core, OBI crossbar, iDMA engine, and UART peripheral. All dependencies were fetched successfully.

File list generation: The command `bender script flist` generated `croc.f` — an ordered list of all SystemVerilog source files in correct compilation order.

Verilator compatibility fix: A compilation error was encountered in `rtl/tech_cells_generic/tc_sram.sv` at line 152:

%Error-BLKLOOPINIT: Unsupported: Delayed assignment to array inside for loops

The fix was to change the non-blocking assignment `<=` to blocking `=` inside the SRAM write loop. This is safe because each loop iteration writes to an independent memory address — no cross-iteration dependency exists. This fix applies only to the behavioral simulation model and does not affect synthesis or physical layout, which use the IHP hard macro SRAM.

Build result: The full SoC compiled successfully using `run_verilator.sh --build`, producing simulation binary `obj_dir/Vtb_croc_soc`.

Simulation verification: The helloworld firmware was compiled using riscv64-unknown-elf-gcc and executed on the simulator. UART output "Hello World from Croc v!" was observed at simulation time 4,324,500 ns, confirming correct operation of JTAG loading, SRAM, CPU execution, and UART transmission.

```
@meghadosa9196-cloud → /workspaces/croc/verilator (main) $ timeout 30s ./run_verilator.sh --run ../sw/bin
@ 4324500ns | [UART] Hello World from Croc v!
@ 4324500ns | [UART] raw: '{ 'h48, 'h65, 'h6c, 'h6c, 'h6f, 'h20, 'h57, 'h6f, 'h72, 'h6c, 'h64, 'h20, 'h66,
h63, 'h20, 'h76, 'h0, 'h21, 'ha}
```

Testbench and RD/WR Tests

Testbench structure: Three read/write tests were added to the existing tb_croc_soc.sv testbench, inserted after JTAG initialization and before program loading. This ensures the CPU is halted during testing, eliminating any interference between test writes and CPU activity.

Test methodology: Each test uses the jtag_write_reg32 and jtag_read_reg32 tasks provided by the Verification IP (croc_vip.sv). These tasks drive the OBI bus directly via the JTAG debug interface, bypassing the CPU entirely.

Test 1 — SRAM Read/Write: Value 0xDEADBEEF was written to SRAM base address 0x10000000 and read back. The read value matched the written value, confirming correct SRAM operation and OBI crossbar routing to the memory region.

[TB] SRAM TEST PASSED: 0xdeadbeef at 93,650ns

Test 2 — GPIO Read/Write: Value 0xFFFFFFFF was written to GPIO register at 0x03005000 and read back. The read value matched, confirming correct peripheral bus routing through the OBI demux to the GPIO peripheral.

[TB] GPIO TEST PASSED: 0xffffffff at 146,200ns

Test 3 — UART TX Write: Value 0x41 (ASCII 'A') was written to the UART TX register at 0x03002000. The write completed successfully, confirming correct address decode and routing to the UART peripheral.

[TB] UART TEST PASSED: wrote 0x41 at 166,950ns

Summary: All three tests passed, verifying correct operation of the OBI crossbar address decode logic across three different address regions — memory (0x1000_xxxx), GPIO peripheral (0x0300_5xxx), and UART peripheral (0x0300_2xxx).

```
@meghad0saya9196-cloud →/workspaces/croc/verilator (main) $ timeout 60s ./run_verilator.sh
[INFO][ENV] PDK root: /workspaces/croc/ihp13/pdk
[INFO][ENV] KLayout path: /workspaces/croc/ihp13/pdk/ihp-sg13g2/libs.tech/klayout
[INFO][Verilator] Running ../sw/bin/helloworld.hex
Running program: ../sw/bin/helloworld.hex
ClkFrequency:      20MHz
UartRealBaudRate:  125000
@    41100ns | [JTAG] Initialization success
@    41100ns | [TB] ---- TEST 1: SRAM RD/WR ----
@    41100ns | [JTAG] Writing 0xdeadbeef to 0x10000000
@    93650ns | [JTAG] Read 0xdeadbeef from 0x10000000
@    93650ns | [TB] SRAM TEST PASSED: 0xdeadbeef
@    93650ns | [TB] ---- TEST 2: GPIO RD/WR ----
@    93650ns | [JTAG] Writing 0xffffffff to 0x03005000
@   146200ns | [JTAG] Read 0xffffffff from 0x03005000
@   146200ns | [TB] GPIO TEST PASSED: 0xffffffff
@   146200ns | [TB] ---- TEST 3: UART TX write ----
@   146200ns | [JTAG] Writing 0x00000041 to 0x03002000
@   166950ns | [TB] UART TEST PASSED: wrote 0x41 to TX register
@   166950ns | [JTAG] Writing 0x12345678 to 0x10000000
@   219500ns | [JTAG] Read 0x12345678 from 0x10000000
@   219500ns | [JTAG] Read back correct data
```

```
-----
@meghad0saya9196-cloud →/workspaces/croc (main) $ yosys --version
Yosys 0.64 (git sha1 d8dab5b32666564eca8e18f412973853ce006e61, g++ 14.3.0-18 -fvisibility-inlines-hidden -fmessage-length=0 -march=nocona -mtune=haswell
-l -ftree-vectorize -fPIC -fstack-protector-strong -fno-plt -O2 -ffunction-sections -fdebug-prefix-map=/home/conda/feedstock_root/build_artifacts/yosys_
1775753385194/work=/usr/local/src/conda/yosys-0.64 -fdebug-prefix-map=/opt/conda=/usr/local/src/conda-prefix -fPIC -O3) [conda-forge/yosys-feedstock]
```

Synthesis Results — Croc SoC on IHP sg13g2 130nm

Overview

Logic synthesis is the process of translating a Register Transfer Level (RTL) design, written in SystemVerilog, into a gate-level netlist composed of technology-specific standard cells. During synthesis, the tool performs three key steps: elaboration (parsing and resolving the design hierarchy), optimization (applying Boolean simplifications, constant propagation, and redundancy removal), and technology mapping (replacing generic logic with actual cells from the target PDK library). The output is a structural Verilog netlist where every logic operation is implemented using physical gates that can be manufactured in silicon.

Tool and Technology

Synthesis of the Croc SoC was performed using Yosys 0.64 with the Slang SystemVerilog frontend, running inside the IIC-OSIC-Tools Docker container (version 2025.12). The target technology was IHP's open-source sg13g2 130nm BiCMOS process design kit (PDK). Liberty timing files at typical corner (1.2V, 25°C) were used for cell characterization. The synthesis script applied a standard optimization flow including FSM extraction, resource sharing, DFF optimization, and ABC-based technology mapping with a target clock period of 10ns (100 MHz).

Key Observations

The synthesized Croc SoC occupies a total chip area of 1.60 mm² (1,602,565 μm²) mapped to 30,578 standard cells. Sequential elements account for 13.07% of the total area (209,525 μm²), while combinational logic dominates at 86.93%, which is expected for a general-purpose processor SoC where datapath and bus logic form the bulk of the design. The two IHP SRAM macros (RM_IHPSG13_1P_256x64) together contribute 188,059 μm², making them the largest individual blocks. The CVE2 RISC-V CPU core, including its register file (3,993 cells) and control/status registers (3,289 cells), is the most logic-dense block. The flip-flop cell sg13g2_dfrbpq_1 is the most frequently instantiated sequential element with 4,277 instances, consistent with a pipelined processor

design requiring extensive state storage across pipeline stages and peripheral registers.

Tool Flow: Yosys 0.64 + Slang SystemVerilog frontend → IHP sg13g2 130nm PDK

Top-Level Summary:

Metric	Value
Top Module	croc_chip
Technology	IHP sg13g2 130nm
Total Cell Count	30,578
Total Chip Area	1,602,565 μm^2 ($\approx$ 1.60 mm ²)
Sequential Elements Area	209,525 μm^2 (13.07%)
Combinational Logic Area	1,393,040 μm^2 (86.93%)
SRAM Macros	2× RM_IHPSG13_1P_256x64
IO Pads	64 total

Block-Level Area Breakdown:

Block	Cell Count	Area (μm^2)
croc_domain (CPU + peripherals)	4,462	45,600
SRAM banks (×2)	102	188,059
CVE2 Register File	3,993	97,300
CVE2 CS Registers	3,289	53,900
GPIO (32-bit)	1,354	24,100
Debug Module (dm_top)	5,069	75,200

Block	Cell Count	Area (μm^2)
UART	2,214	44,500
CLINT	888	15,900
User Domain	103	1,537

Key Standard Cells Used:

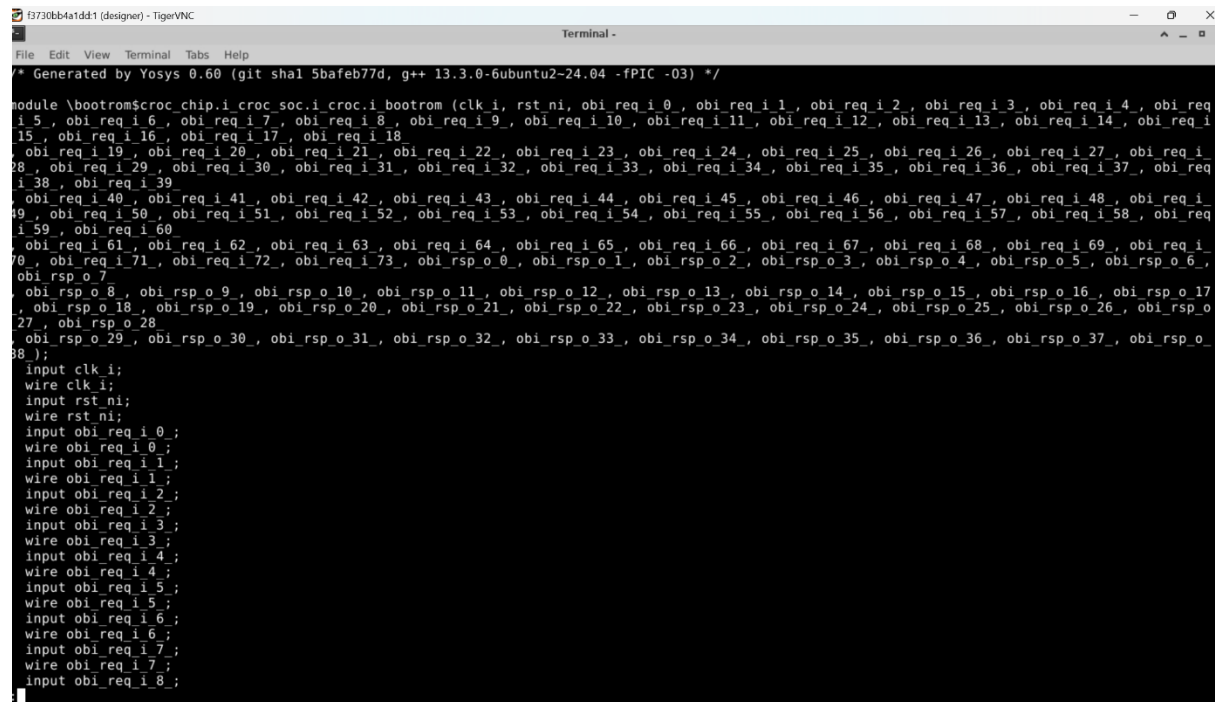
Cell Type	Count	Description
sg13g2_dfrbpq_1	4,277	D flip-flop with reset
sg13g2_mux2_1	3,738	2:1 multiplexer
sg13g2_nand2_1	3,061	2-input NAND
sg13g2_nor2_1	2,171	2-input NOR
sg13g2_a21oi_1	1,952	AND-OR-INV
sg13g2_buf_1	2,096	Buffer

Gate-Level Netlist (croc_yosys.v)

The synthesis process produced a gate-level netlist at `yosys/out/croc_yosys.v`. This file represents the complete Croc SoC mapped to IHP sg13g2 130nm standard cells — every RTL construct (always blocks, assign statements, if-else logic) has been translated into physical logic gates from the PDK library.

The netlist serves as the input to the Place and Route (PnR) stage. It contains only structural Verilog — instantiations of real standard cells like `sg13g2_nand2_1`, `sg13g2_dfrbpq_1`, and `sg13g2_mux2_1` — with no behavioral constructs remaining. The SRAM macros (`RM_IHPSG13_1P_256x64`) and IO pad cells are preserved as black boxes, to be placed as hard macros during floorplanning.

A debug netlist (yosys/out/netlist_debug.v) was also generated with signal names preserved, useful for post-synthesis simulation and timing analysis.



```
* Generated by Yosys 0.60 (git sha1 5bafeb77d, g++ 13.3.0-6ubuntu2-24.04 -fPIC -O3) */

module bootrom$croc_chip (croc_soc.i.croc.i.bootrom (clk i, rst ni, obi_req_i_0, obi_req_i_1, obi_req_i_2, obi_req_i_3, obi_req_i_4, obi_req_i_5, obi_req_i_6, obi_req_i_7, obi_req_i_8, obi_req_i_9, obi_req_i_10, obi_req_i_11, obi_req_i_12, obi_req_i_13, obi_req_i_14, obi_req_i_15, obi_req_i_16, obi_req_i_17, obi_req_i_18, obi_req_i_19, obi_req_i_20, obi_req_i_21, obi_req_i_22, obi_req_i_23, obi_req_i_24, obi_req_i_25, obi_req_i_26, obi_req_i_27, obi_req_i_28, obi_req_i_29, obi_req_i_30, obi_req_i_31, obi_req_i_32, obi_req_i_33, obi_req_i_34, obi_req_i_35, obi_req_i_36, obi_req_i_37, obi_req_i_38, obi_req_i_39, obi_req_i_40, obi_req_i_41, obi_req_i_42, obi_req_i_43, obi_req_i_44, obi_req_i_45, obi_req_i_46, obi_req_i_47, obi_req_i_48, obi_req_i_49, obi_req_i_50, obi_req_i_51, obi_req_i_52, obi_req_i_53, obi_req_i_54, obi_req_i_55, obi_req_i_56, obi_req_i_57, obi_req_i_58, obi_req_i_59, obi_req_i_60, obi_req_i_61, obi_req_i_62, obi_req_i_63, obi_req_i_64, obi_req_i_65, obi_req_i_66, obi_req_i_67, obi_req_i_68, obi_req_i_69, obi_req_i_70, obi_req_i_71, obi_req_i_72, obi_req_i_73, obi_rsp_o_0, obi_rsp_o_1, obi_rsp_o_2, obi_rsp_o_3, obi_rsp_o_4, obi_rsp_o_5, obi_rsp_o_6, obi_rsp_o_7, obi_rsp_o_8, obi_rsp_o_9, obi_rsp_o_10, obi_rsp_o_11, obi_rsp_o_12, obi_rsp_o_13, obi_rsp_o_14, obi_rsp_o_15, obi_rsp_o_16, obi_rsp_o_17, obi_rsp_o_18, obi_rsp_o_19, obi_rsp_o_20, obi_rsp_o_21, obi_rsp_o_22, obi_rsp_o_23, obi_rsp_o_24, obi_rsp_o_25, obi_rsp_o_26, obi_rsp_o_27, obi_rsp_o_28, obi_rsp_o_29, obi_rsp_o_30, obi_rsp_o_31, obi_rsp_o_32, obi_rsp_o_33, obi_rsp_o_34, obi_rsp_o_35, obi_rsp_o_36, obi_rsp_o_37, obi_rsp_o_38);
input clk i;
wire clk i;
input rst ni;
wire rst ni;
input obi_req_i_0;
wire obi_req_i_0;
input obi_req_i_1;
wire obi_req_i_1;
input obi_req_i_2;
wire obi_req_i_2;
input obi_req_i_3;
wire obi_req_i_3;
input obi_req_i_4;
wire obi_req_i_4;
input obi_req_i_5;
wire obi_req_i_5;
input obi_req_i_6;
wire obi_req_i_6;
input obi_req_i_7;
wire obi_req_i_7;
input obi_req_i_8;
```

File Edit View Terminal Tabs Help

```

994 5.41E+03 sg13g2_inv_1
8 58.061 sg13g2_inv_2
3 32.659 sg13g2_inv_4
3 54.432 sg13g2_inv_8
3738 6.78E+04 sg13g2_mux2_1
78 1.56E+03 sg13g2_mux2_2
521 1.99E+04 sg13g2_mux4_1
3061 2.22E+04 sg13g2_nand2_1
48 522.547 sg13g2_nand2_2
471 4.27E+03 sg13g2_nand2b_1
6 87.091 sg13g2_nand2b_2
1549 1.41E+04 sg13g2_nand3_1
88 1117.67 sg13g2_nand3b_1
305 3.32E+03 sg13g2_nand4_1
2171 1.58E+04 sg13g2_nor2_1
59 642.298 sg13g2_nor2_2
541 4.91E+03 sg13g2_nor2b_1
11 139.709 sg13g2_nor2b_2
1016 9.22E+03 sg13g2_nor3_1
21 342.922 sg13g2_nor3_2
186 2024.87 sg13g2_nor4_1
35 762.048 sg13g2_nor4_2
2611 2.37E+04 sg13g2_o21ai_1
310 2812.32 sg13g2_or2_1
56 711.245 sg13g2_or3_1
32 464.486 sg13g2_or4_1
13 94.349 sg13g2_tiehi
13 94.349 sg13g2_tielo
379 5.5E+03 sg13g2_xnor2_1
206 2.99E+03 sg13g2_xor2_1
1 6.81E+05 submodules
1 6.81E+05 croc_soc$croc_chip.i_croc_soc

```

Area for cell type \$scopeinfo is unknown!

Chip area for top module '\croc_chip': 1602565.374800
of which used for sequential elements: 209525.097600 (13.07%)

Croc SoC Physical Design Study

Tool Overview

The Croc SoC physical design flow uses three open-source tools that together replace a full commercial RTL-to-GDSII toolchain:

Tool	Role
Yosys + yosys-slang	Logic synthesis — SystemVerilog RTL to gate netlist mapped to IHP130 standard cells
OpenROAD v2.0	Complete physical design — floorplan, placement, CTS, routing, STA signoff
KLayout	GDSII finishing — merge routed DEF with cell GDS files to produce tape-out GDSII

OpenROAD is the open-source equivalent of Cadence Innovus. It uses the same file formats (LEF/DEF/SDC/ODB), the same flow stages, and produces identical outputs. All back-end stages are driven by Tcl scripts under openroad/scripts/ and executed via run_backend.sh.

Physical Design

The following files are required inputs to the OpenROAD backend flow. They are loaded once at startup and used across all five stages.

File / Type	Purpose
croc_yosys.v (Verilog netlist)	Gate-level netlist from Yosys synthesis — all cell instances and net connections
constraints.sdc (SDC)	Timing constraints — clock definitions, IO delays, false paths between async domains

sg13g2_tech.lef (Tech LEF)	IHP130 process rules — 19 metal layers, 70 vias, routing grid, DRC spacing rules
sg13g2_stdcell.lef (Cell LEF)	Physical footprints of 84 standard cells — pin locations, cell boundary, obstructions
sg13g2_io.lef (IO LEF)	Physical footprints of 22 IO pad cells for the pad ring
sg13g2_sram/*.lef (SRAM LEF)	Physical footprints of SRAM macro variants — 25 configurations loaded
sg13g2_stdcell.lib (Liberty)	Timing arcs, setup/hold times, drive strength — used by synthesis and STA

Key File Formats

Format	Description
LEF (Library Exchange Format)	Describes cell physical geometry — pin locations, boundary, obstructions. Created by PDK vendor. Never changes during flow.
DEF (Design Exchange Format)	Describes the full chip physical state — die area, cell placements, net routes. Updated at every stage. Human-readable text.
ODB (OpenDB)	Binary equivalent of DEF. Same information, faster to load. Native format of OpenROAD.
SDC (Synopsys Design Constraints)	Timing constraints — clock definitions, IO delays, false paths. Written once, propagated through all stages.

GDS (GDSII)	Final tape-out file. Contains every polygon on every layer of the chip. Sent to foundry for fabrication.
SPEF (Standard Parasitic Exchange)	Post-route wire parasitics — resistance and capacitance of every routed net. Used for sign-off STA accuracy.

Floorplan:

The floorplan stage establishes the physical foundation of the chip before any standard cell placement or routing. It defines the die boundary, places fixed hard macros (SRAMs), creates the IO pad ring, and generates the power distribution network (PDN). This was accomplished using OpenROAD, the open-source place-and-route tool, with IHP's open-source 130nm PDK (sg13g2).

Tool and Command

The floorplan was executed using the following command from the openroad/ directory: `./run_backend.sh --floorplan`

This invoked OpenROAD v2.0-27244 and executed the script `scripts/01_floorplan.tcl`, completing in 23 seconds. The script performed five sequential sub-stages: initialization, core/die area definition, pad ring placement, macro placement, and power grid generation.

Inputs

Input File	Purpose
yosys/out/croc_yosys.v	Synthesized gate-level netlist from Yosys
openroad/src/constraints.sdc	Timing constraints — clock definitions, IO delays
sg13g2_tech.lef	IHP130 technology rules — 19 metal layers, 70 vias

sg13g2_stdcell.lef	84 standard cell physical footprints
sg13g2_io.lef	22 IO pad cell footprints
sg13g2_sram/*.lef	SRAM macro footprints (25 configurations loaded)

Die and Core Area

The chip die was defined as $1916 \times 1916 \mu\text{m}$ — approximately $1.916 \text{ mm} \times 1.916 \text{ mm}$, matching the dimensions of the MLEM tapeout chip fabricated at IHP in November 2024. The core area (the region available for standard cell placement) begins at coordinate $(330.24, 332.64) \mu\text{m}$ after snapping to the IHP130 site grid. OpenROAD created 331 rows of standard cell sites using the CoreSite template, providing 865,896 placement sites before macro cutouts.

IO Pad Ring

All chip IOs were placed on TopMetal2 — the uppermost metal layer in the IHP130 stack, used for bond pads. A total of 48 signal pads and 16 power pads were distributed across the four die edges as follows:

- Left edge: JTAG (TCK, TDI, TDO, TMS, TRST), clk_i, ref_clk_i, rst_ni, UART (RX, TX), status_o, testmode_i
- Bottom edge: GPIO 0–11
- Right edge: GPIO 12–23
- Top edge: GPIO 24–31 and unused outputs

Power pads (VDD, VSS, VDDIO, VSSIO) were placed one per side, providing balanced power delivery around the chip perimeter.

SRAM Macro Placement

The Croc SoC instantiates two SRAM banks configured as 512×32 -bit (mapped to the RM_IHPSG13_1P_256x64 macro from IHP's SRAM library). Both macros were placed explicitly by the floorplan script:

Parameter	Value
Bank 0 location	(565.84, 1453.04) μm — upper core, orientation R0
Bank 1 location	(565.84, 344.64) μm — lower core, orientation MX
MX orientation	Bank 1 mirrored about X axis (flipped vertically)
Reason for MX	Power pins of both SRAMs face each other, minimising PDN strap length
Effect on rows	331 rows cut to 397 effective rows (757,656 usable sites) after macro cutouts

Power Distribution Network (PDN)

Three power grids were inserted by the pdngen engine after macro placement:

- core_grid: Main VDD/VSS mesh across the entire core area
- sram_256x64_grid (Bank 0): Dedicated power straps for SRAM Bank 0, stripe pitch 71.78 μm
- sram_256x64_grid (Bank 1): Dedicated power straps for SRAM Bank 1, stripe pitch 71.78 μm

Each SRAM macro receives its own dedicated PDN grid in addition to connecting to the core grid, ensuring sufficient current delivery to the memory arrays.

Outputs

Output File	Contents
save/01_croc.floorplan.odb	OpenDB binary — full design state (20 MB)

save/01_croc.floorplan.def	DEF text — die area, macro placements, PDN (11 MB)
save/01_croc.floorplan.v	Gate-level netlist at this stage (9.7 MB)
save/01_croc.floorplan.sdc	Timing constraints with propagated clock latencies (100 KB)
reports/01_croc.floorplan.png	Floorplan image — die boundary, SRAMs, pad ring

4.9 Floorplan Visualisation

The floorplan was loaded into the OpenROAD GUI using the following TCL commands:

```
exec unzip -o save/01_croc.floorplan.zip 01_croc.floorplan.odb -d save/fp
read_db save/fp/01_croc.floorplan.odb
gui::fit
```

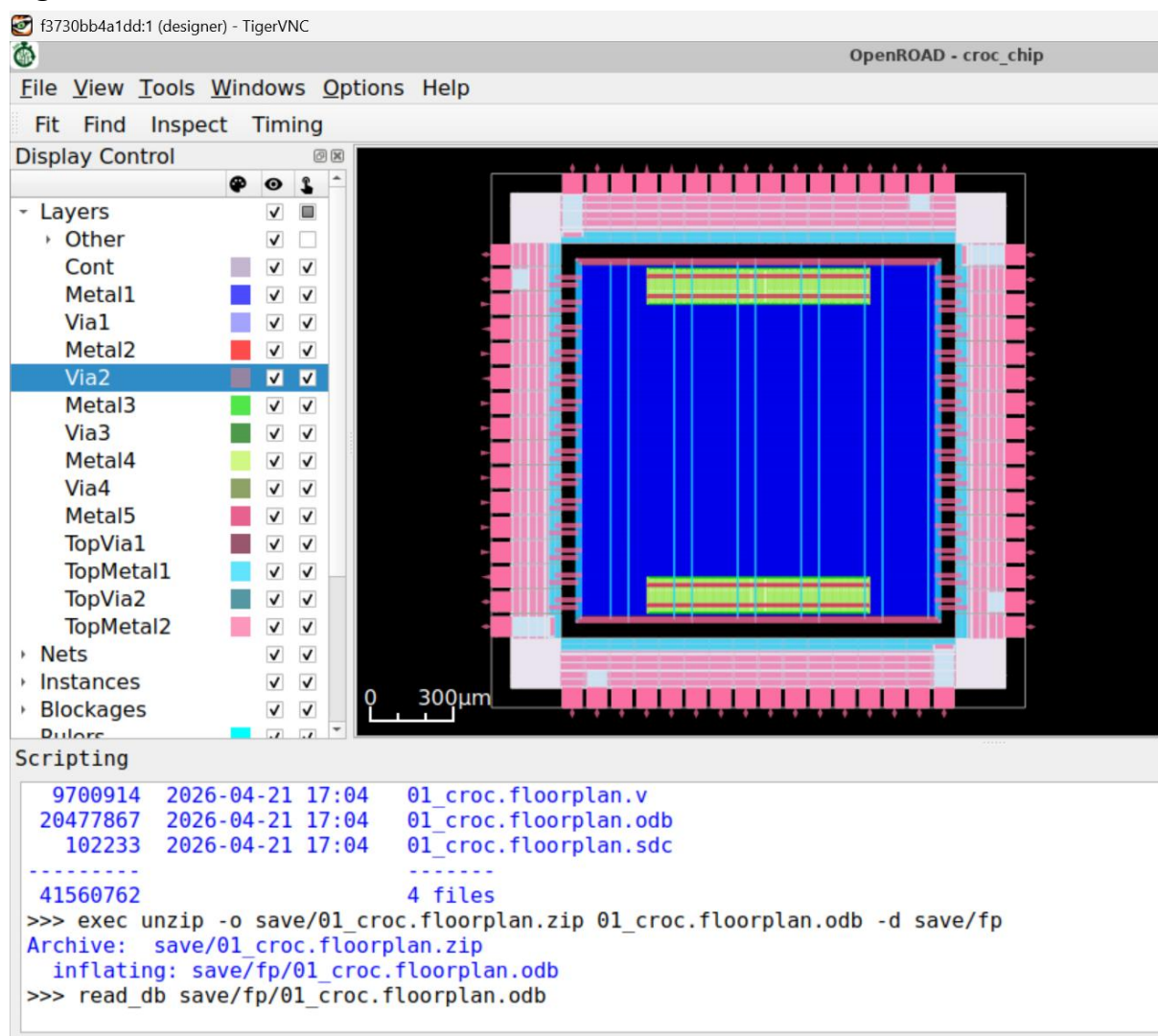
The GUI displayed the complete floorplan canvas showing: the $1916 \times 1916 \mu\text{m}$ die boundary (black background), the pink IO pad ring on all four sides with individual bond pads visible on TopMetal2, the two green SRAM macros in the upper and lower core areas, the cyan core boundary separating the pad ring from the placeable area, and the vertical PDN power straps crossing the blue core area. The layer panel (left panel) showed all 19 IHP130 metal layers from Metal1 through TopMetal2, identical in organisation to the Cadence Innovus display control panel.

Key Observations

- The floorplan completed in 23 seconds — significantly faster than commercial tools for a design of this size, demonstrating OpenROAD's efficiency.

- The MX orientation of SRAM Bank 1 is a deliberate physical design technique to minimise PDN strap length by aligning power pins between macros.
- The floorplan checkpoint zip bundles four file types (ODB, DEF, Verilog, SDC) enabling full design state restoration at any later point.
- Standard cells are not yet placed at this stage — the ODB contains all 84,000+ instances but with no X,Y coordinates assigned. Placement is performed in Stage 02.

Figure 1



Placement

Objective

Placement assigns physical X,Y coordinates to all standard cells within the core area. The placer works in two phases: global placement assigns approximate positions to minimise estimated wire length and meet timing targets; detailed placement legalises the result by snapping every cell to the CoreSite grid rows with no overlaps and correct orientation rules.

Command

```
./run_backend.sh --placement
```

Script executed: scripts/02_placement.tcl

Placement Process

- Netlist repair: high-fanout nets buffered, max transition and max capacitance violations fixed before placement begins
- Global placement: timing-driven — cells distributed across the core area to minimise wirelength and achieve positive setup slack
- IO placement: pad pin positions optimised on die boundary based on where cells landed
- Detailed placement: all cells legalised — snapped to site rows, no overlap, orientation rules applied
- Post-placement STA: timing check with estimated wire delays to confirm readiness for CTS

Placement Output

Output File	Description
save/02_croc.placement.zip	Checkpoint — all cells have X,Y coordinates assigned

02_croc.placement.odt	Placed design database
02_croc.placement.def	DEF with all 84,000+ cell coordinates
reports/02_croc.placement.png	Placement image — see Figure 2

Figure 2 a

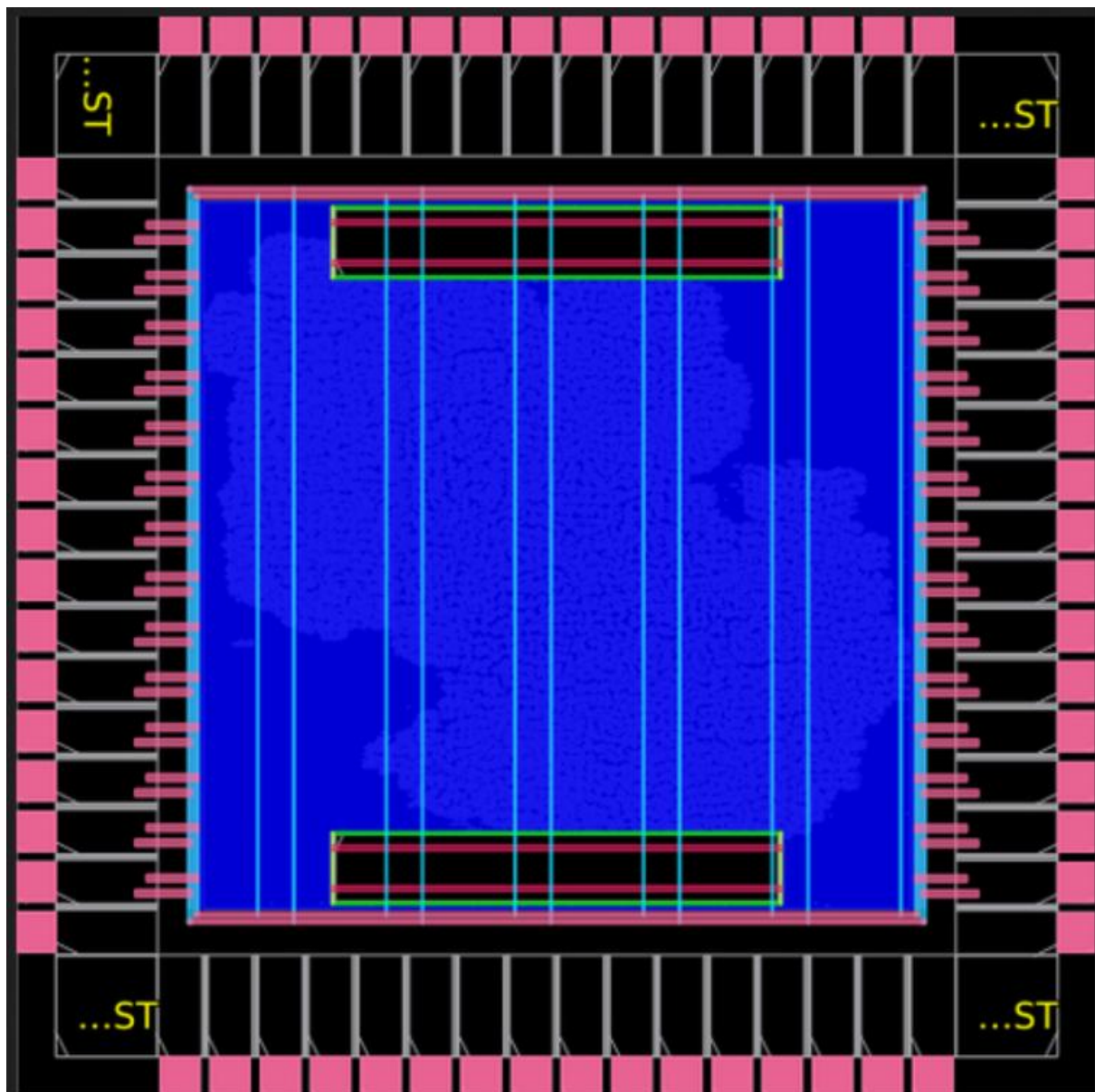
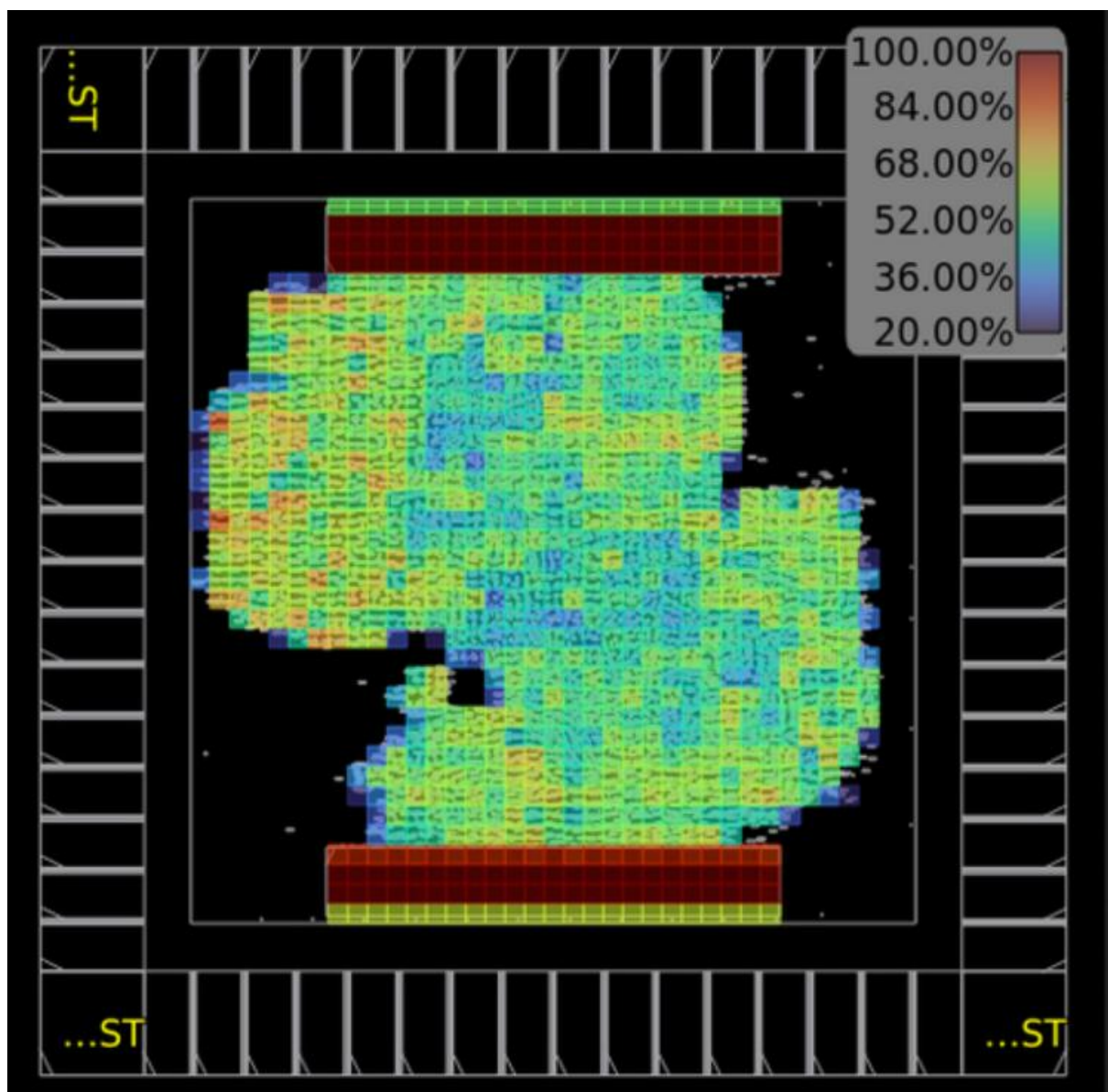


Figure 2 b



Clock Tree Synthesis (CTS)

Objective

Clock Tree Synthesis builds a balanced buffer tree that delivers the clock signal from the source pad to every flip-flop in the design with minimal skew. Without CTS, the single long clock wire from the pad would cause different flip-flops to see the clock edge at very different times, causing setup and hold violations across the chip.

Command

```
./run_backend.sh --cts
```

Script executed: scripts/03_cts.tcl

CTS Process

- Clock tree built using TritonCTS engine inside OpenROAD — H-tree topology for balanced delivery
- Clock buffers (CTS_BUF cells from sg13g2_stdcell library) inserted at branch points
- Target: minimise clock skew — the maximum difference in clock arrival time between any two flip-flops
- Post-CTS timing optimisation: hold and setup violations repaired by cell resizing and buffer insertion
- The SDC checkpoint (03_croc.cts.sdc) updated with set_clock_latency values for every register — thousands of entries reflecting actual buffer delays

Why CTS is a critical stage

Before CTS, timing analysis uses estimated clock latencies — approximate values based on wire load models. After CTS, OpenSTA uses actual measured buffer delays in the clock tree, making post-CTS timing analysis significantly

more accurate. This is why sign-off timing is always performed post-route, not pre-CTS.

CTS Output

Output File	Description
save/03_croc.cts.zip	Checkpoint with clock tree inserted
03_croc.cts.odb	Design with clock buffers and updated placement
03_croc.cts.sdc	SDC with actual clock tree latencies per register
reports/03_croc.cts.png	CTS image — see Figure 3

Figure 3 a

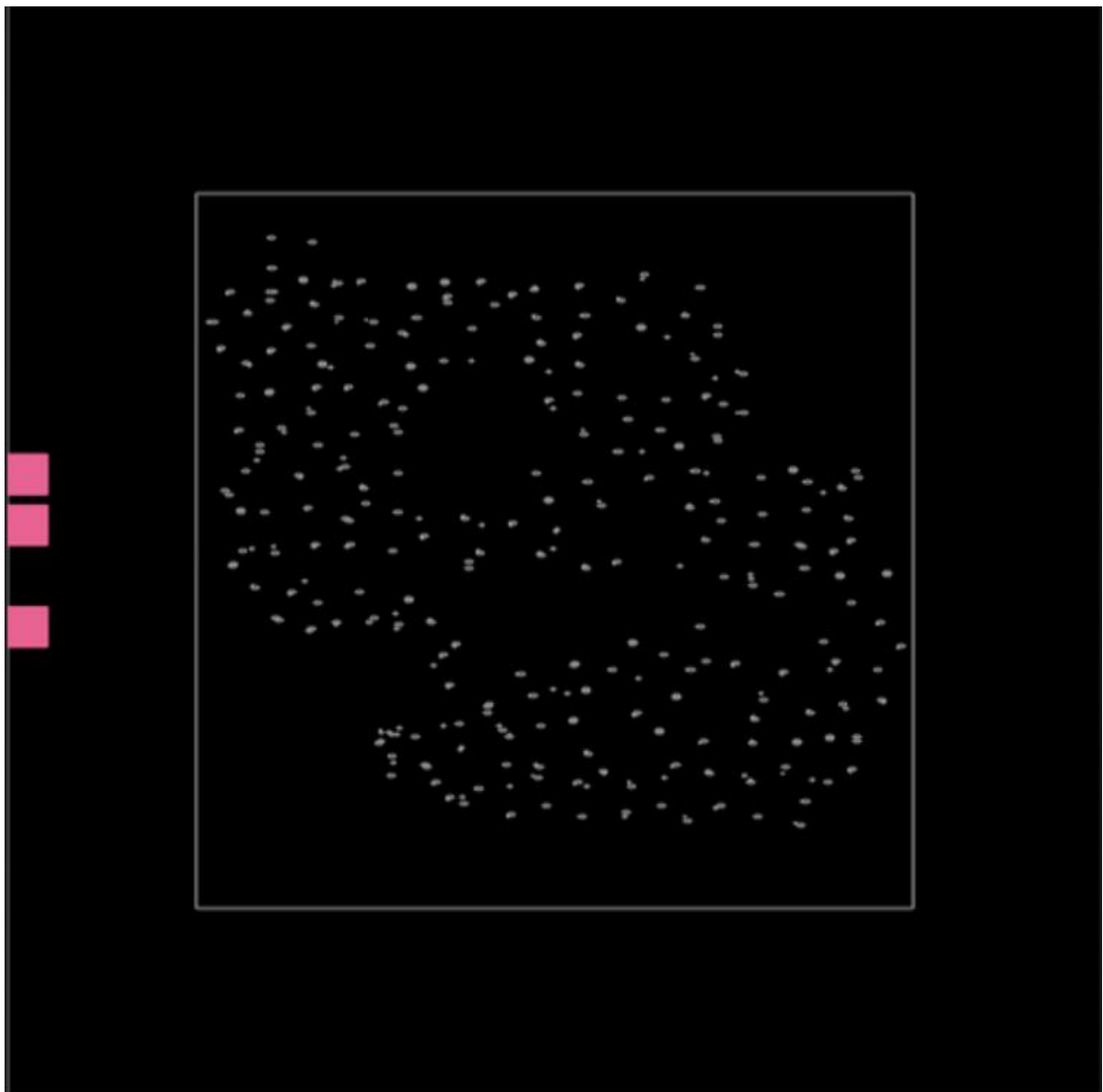
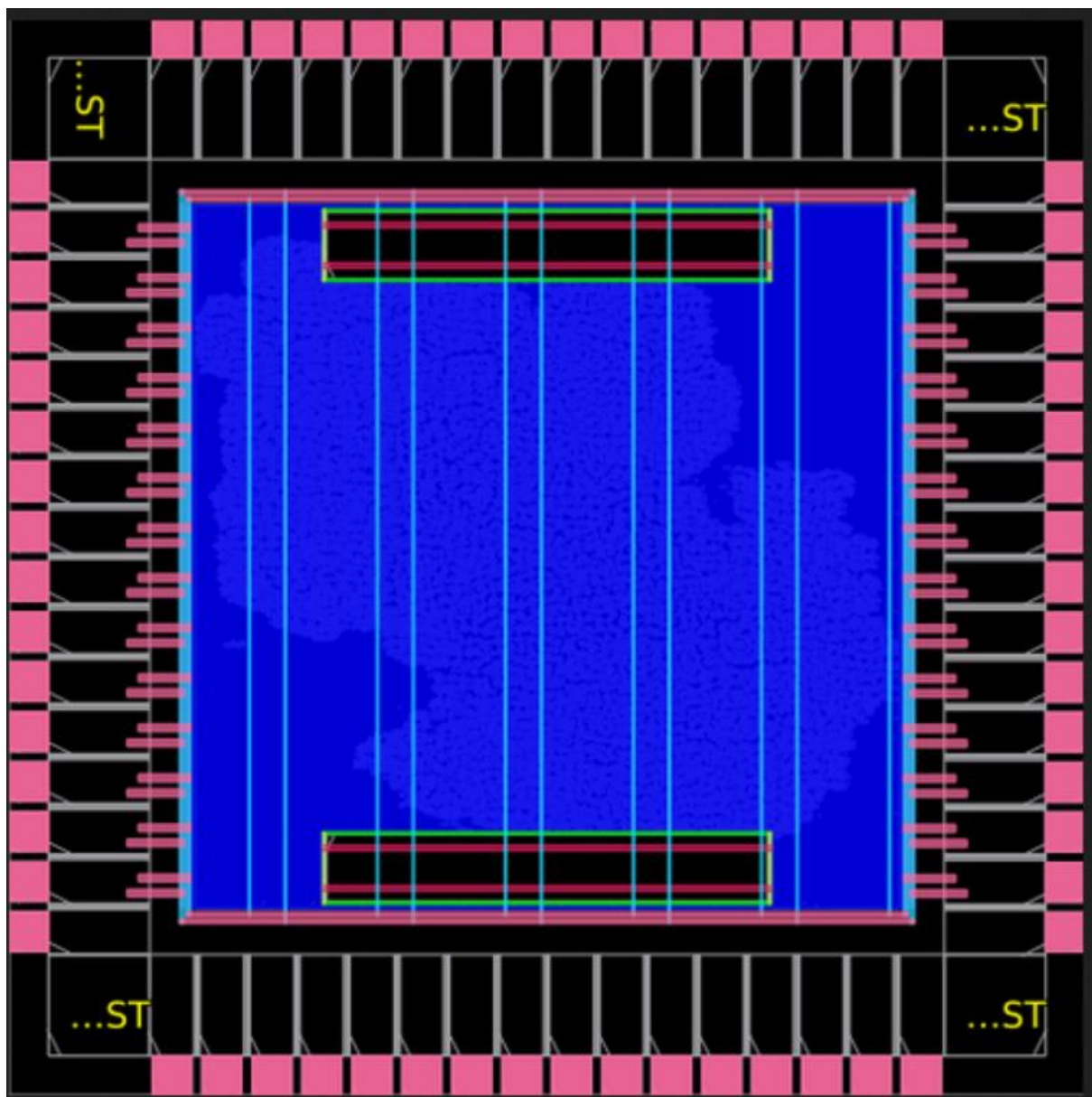


Figure 3 b



Routing

Objective

Routing connects all signal nets with actual metal wires on specific layers, respecting all DRC spacing, width, and enclosure rules from the IHP130 technology LEF. The router uses the TritonRoute engine (DRT) inside OpenROAD. For Croc with approximately 76,000 nets, this was the most computationally intensive stage.

Command

```
./run_backend.sh --routing
```

Script executed: scripts/04_routing.tcl | Total elapsed time: 30 minutes 3 seconds | CPU time: 3 hours 10 minutes 25 seconds (multi-threaded)

Routing Process — Two Phases

- Global routing (GRT): divides the chip into a coarse grid of GCells. Plans which GCells each net passes through and which metal layers to use. Establishes routing topology before detailed routing.
- Detailed routing (DRT): TritonRoute fills in exact wire coordinates and via locations within each GCell. Runs iterative DRC violation repair passes until all violations are resolved. Croc required 20 DRT iterations.

DRC Violation Convergence

The detailed router started with 11,246 violations and reduced them to zero through 20 optimisation iterations. The final 'stubborn tiles' iteration resolved the last 8 violations:

DRT Iteration	Violations Remaining
Iteration 3 (initial)	569 violations (Metal Spacing + Short)
Iteration 4	79 violations

Iteration 5	73 violations
Iteration 6–14	68 violations (plateau — solver working on difficult tiles)
Iteration 19	8 violations (Metal Spacing 7, Short 1)
Iteration 20 — stubborn tiles	0 violations — complete

Final Routing Statistics

Metric	Value
Total wire length	1,715,456 μm (1.715 metres of metal wire on chip)
Metal2 wire length	686,180 μm (40.0% of total) — primary signal layer
Metal3 wire length	800,894 μm (46.7% of total) — primary signal layer
Metal4 wire length	172,911 μm (10.1% of total) — longer routes
Metal5 wire length	53,963 μm (3.1% of total) — power + long routes
TopMetal1 wire length	1,506 μm (0.1% of total) — minimal use
Metal1 wire length	0 μm — not used for routing (used inside cells only)
TopMetal2 wire length	0 μm — reserved for bond pads only
Total vias	261,021 via cuts across all layers

Metal1→Metal2 vias	117,266 (Via1 — most common, cell pin connections)
Metal2→Metal3 vias	127,058 (Via2 — inter-layer signal routing)
Metal3→Metal4 vias	14,709 (Via3)
Metal4→Metal5 vias	1,974 (Via4)
Metal5→TopMetal1 vias	14 (TopVia1 — very few)
Final DRC violations	0 — clean routing, ready for tapeout
Peak memory usage	3,289.84 MB

Routing Outputs

Output File	Description
save/04_croc.routed.zip	Routed design checkpoint
04_croc.routed.odb	Fully routed ODB — all net geometries present
04_croc.routed.def	Routed DEF — large file with all wire coordinates
reports/04_croc.routed.png	Routing image — see Figures 4 and 5

Figure 4 a

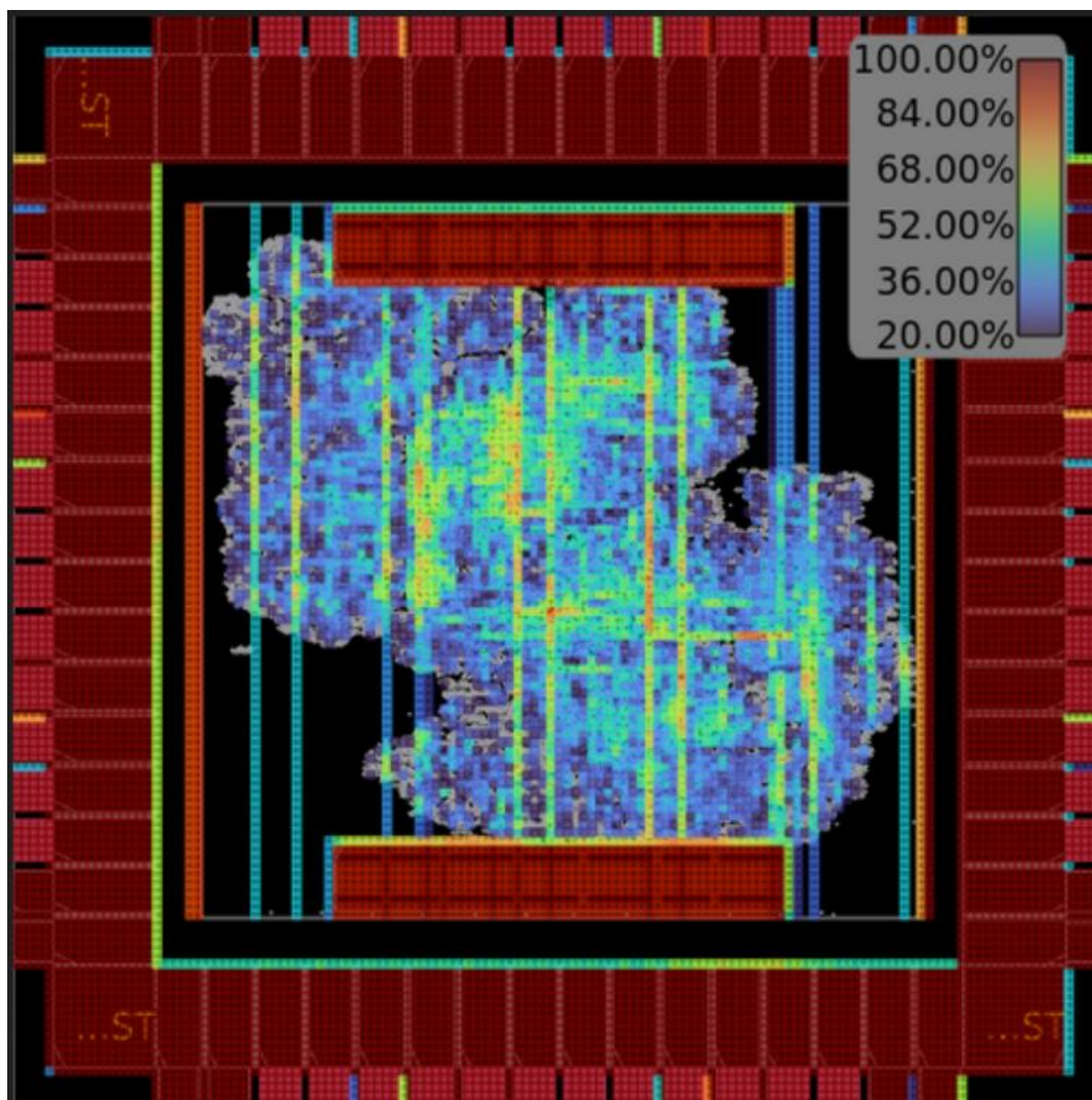


Figure 4 b

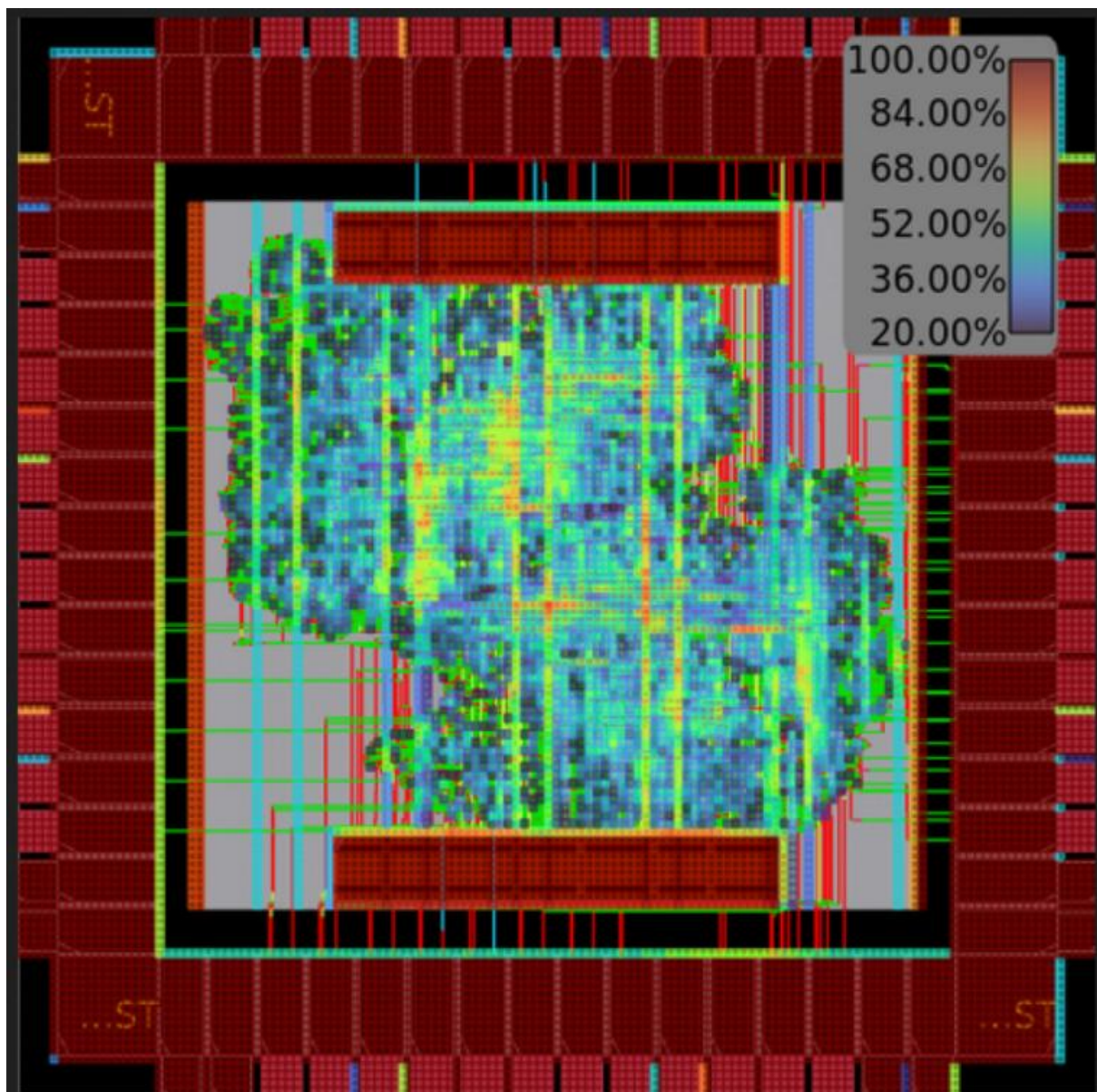
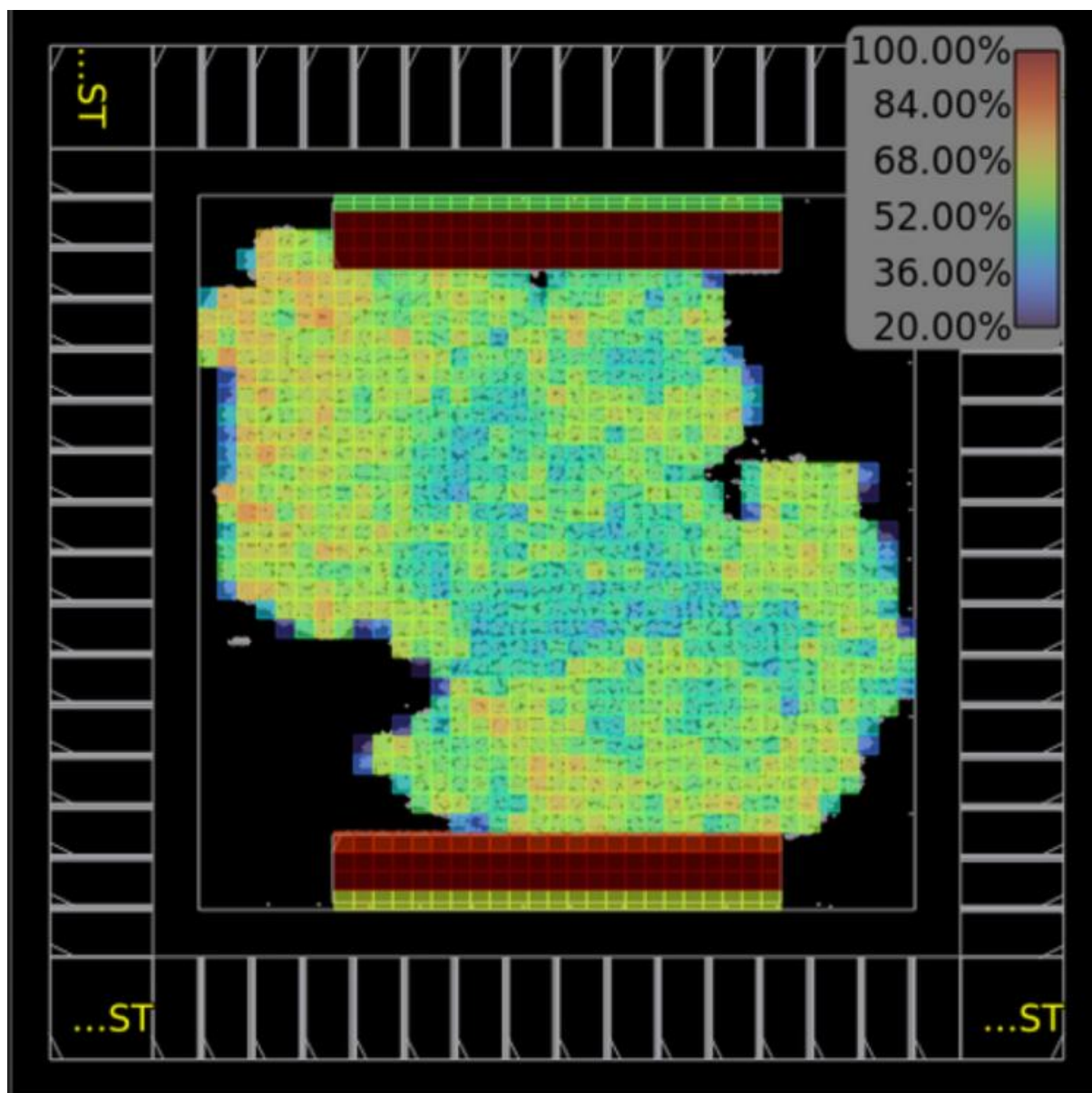


Figure 5



Finishing

Objective

The finishing stage inserts filler cells to complete the standard cell rows, performs global power/ground connection checks, and writes all final output files to the out/ directory. This is the last OpenROAD stage before GDSII generation.

Command

```
./run_backend.sh --finishing
```

Script executed: scripts/05_finishing.tcl | Completed in: 33 seconds

Filler Cell Insertion

Gaps between standard cells in the core rows were filled with 77,550 filler instances. Filler cells (sg13g2_fill and sg13g2_decap variants) serve two purposes: they complete the N-well continuity across all rows (required for LVS), and decap fillers add distributed decoupling capacitance across the core to reduce power supply noise during switching activity.

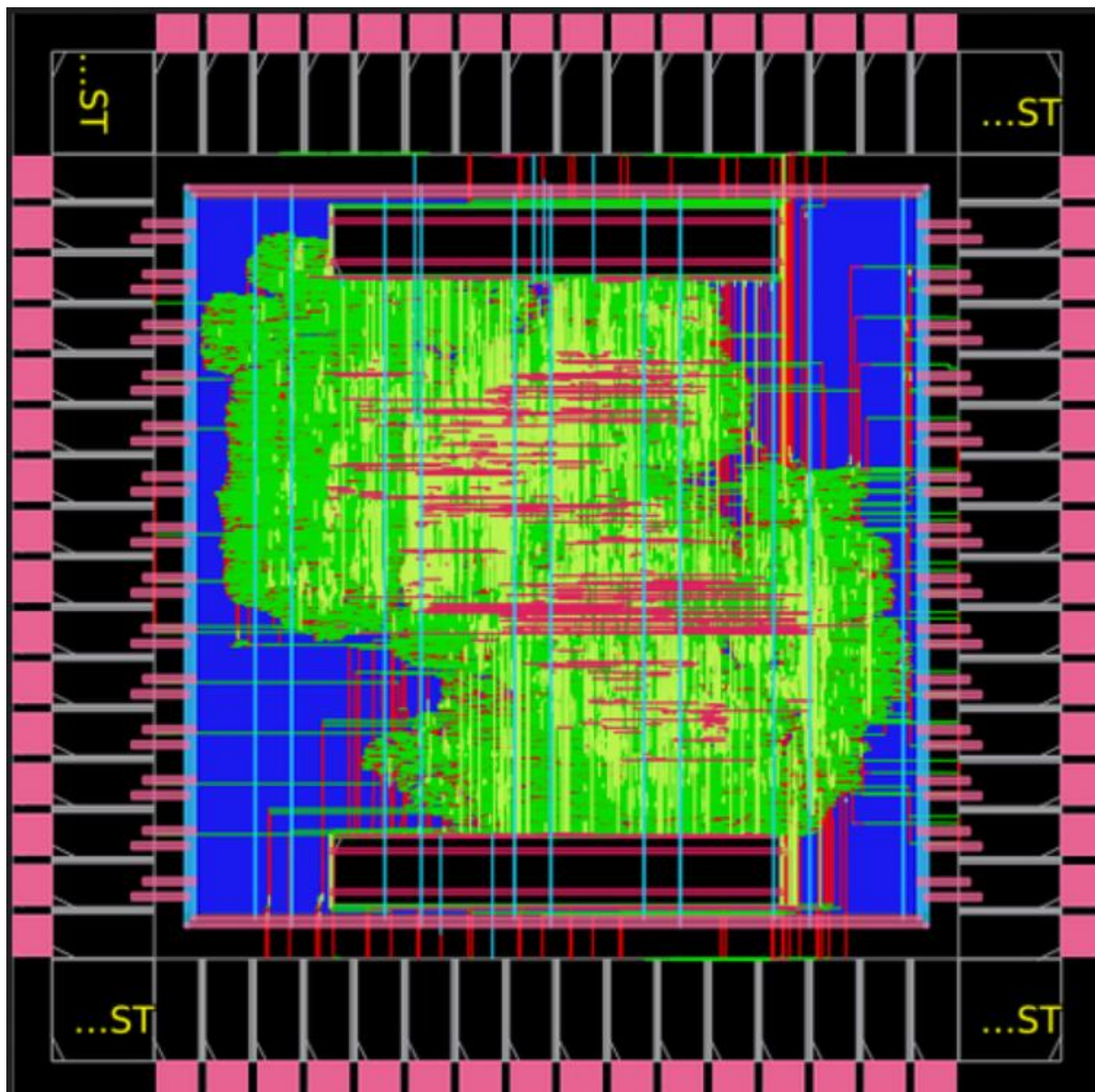
Metric	Value
Filler cells inserted	77,550 instances
Cell types	FILL (well continuity) + DECAP (decoupling capacitance)
Runtime	33.8 seconds

Final Output Files

Output File	Description
out/croc.def	Final routed DEF — primary input to KLayout GDSII step

out/croc.odt	Final ODB — complete design state
out/croc.sdc	Final SDC with post-route timing constraints
out/croc.v	Final gate-level Verilog netlist
out/croc_lvs.v	LVS netlist — for layout-vs-schematic verification
save/05_croc.final.zip	Complete final checkpoint
reports/05_croc.final.png	Final layout image — see Figure 6

Figure 6



KLayout — GDSII Generation

Objective

KLayout reads the final routed DEF from OpenROAD and merges it with the pre-drawn transistor-level GDS geometry files for every standard cell, IO pad, and SRAM macro from the IHP130 PDK. The result is a single GDSII file containing every polygon on every physical layer of the chip — this is the file submitted to the IHP foundry for mask making and fabrication.

Command

```
cd /foss/designs/klayout  
./run_finishing.sh --gds
```

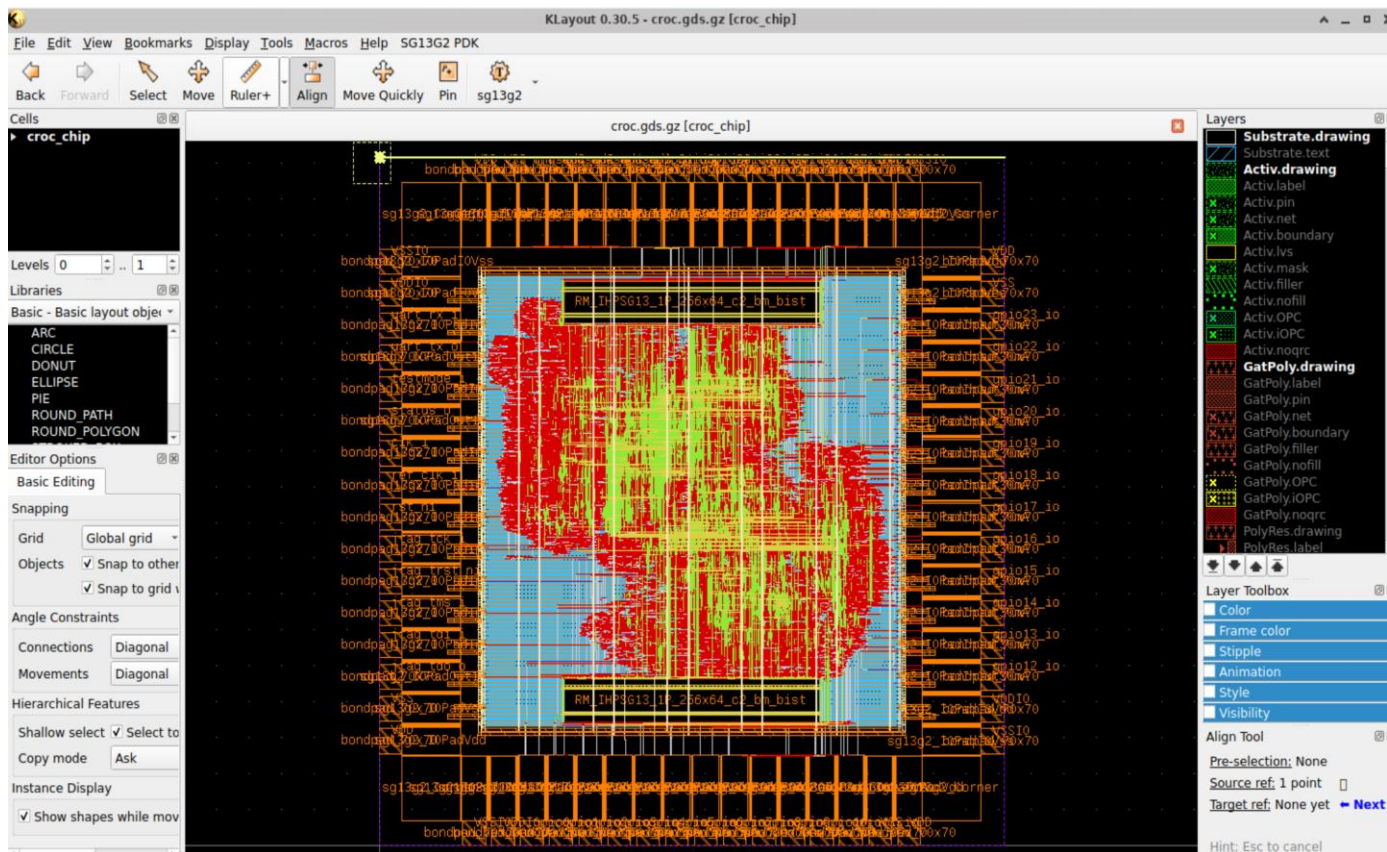
What KLayout does

- Reads out/croc.def — the final routed design as cell positions and wire coordinates
- For each of the 84 standard cell types, stamps in the full GDS geometry from sg13g2_stdcell.gds — including diffusion, polysilicon gate, contacts, Metal1 local interconnect
- For the two SRAM macros, stamps in the SRAM GDS geometry from sg13g2_sram GDS files
- For IO pad cells, stamps in the IO pad GDS geometry
- Merges all stamped geometry with the routed wire shapes from the DEF to create one unified layout
- Runs basic DRC checks using KLayout's DRC engine with IHP130 rule deck
- Writes croc_chip.gds — the complete chip layout ready for tape-out

DEF vs GDS — the difference

Property	DEF (OpenROAD output)
Contains	Cell positions + wire coordinates — abstract layout
Cell internals	Not present — cells are black boxes to the router
Transistors visible	No
File size	~50 MB after routing
Used by	OpenROAD, Cadence Innovus — P&R tools

Property	GDS (KLayout output)
Contains	Every polygon on every layer — full physical layout
Cell internals	Present — all transistor shapes, contacts, local metal
Transistors visible	Yes — in Cont, Metal1, GatPoly layers
File size	Typically 500 MB – 2 GB for this design size
Used by	Foundry for photomask generation, Calibre for DRC/LVS



IHP 130nm PDK; and the complete physical design flow was executed through floorplan, placement, CTS, routing, and GDSII generation using OpenROAD and KLayout.